CS 410 Binary to Assembly Activity

Andres Trujillo
May 24, 2026
Module Three

File One: assignment3_1.o

Functions	Blocks of Assembly Code	Explanation of Functionality

main	push %rbp mov %rsp, %rbp mov $0x400634, %edi	The current stack pointer becomes the base of the new stack frame. The base pointer register becomes the a fixed reference point for local variables and arguments. Then it loads the address $0x400634 into the 32-bit Extended Destination index register(%edi). This register typically points to the destination buffer for string operations. Also, can be used for general-purpose index register in addressing modes.

main	call 400450 <puts@plt>	400450 is the memory address of a PLT entry (Procedure linkage table) or a jump table entry used for dynamic linking. It needs to call it this way using the string within the edi register because it's dynamically linked and the program which is located on the stack that could be any x86-64 system that is capable of running the process will be calling it through the global offset table for the C runtime library to actually get the function so it may display on the systems screen.

main	mov $0x40048, %edi call 400450 <puts@plt> mov $0x40065c, %edi call 400450 <puts@plt>	This performs the same thing as above while moving the edi register through the addresses of the string locations while calling the puts function to display the characters.

	mov $0x0, %edi call 400480 <exit@plt> nopl 0x0(%rax)	This follows the same System V ABI convention inserting the first argument into the edi register of zero. Then returns the exit code of the edi variable. And this internally flushes buffers, runs cleanup functions, and terminates program immediately.
	String contents: Ship to: John Smith 123 Los Angeles Rd. Los Angeles, CA 90025 String memory address: 0x00400630 to 0x00400670	

File Two: assignment3_2.o

Functions	Blocks of Assembly Code	Explanation of Functionality
main	push %rbp mov %rsp, %rbp	Saves the callers base pointer on the stack to return to the C runtime and creates a base pointer to offset into the function.
main	sub $0x20, %rsp	Subtracts 32 bytes from the stack pointer for local variables.
main	mov %fs:0x28, %rax mov %rax, -0x8(%rbp)	Loads the stack canary from FS segment offset 0x28 and saves into the stack frame at [rbp - 8] to check for buffer overflows.
main	lea -0x20(%rbp), %rax	It means to load effective address at negative 0x20 offset of the %rbp register which is our local buffer set aside from the developers code.
main	mov %rax,%rsi mov $0x40072b,%edi mov $0x0,%eax call __isoc99_scanf@plt	Then creates two function arguments at %edi and %eax, and calls scanf. When the function "scanf" takes two arguments we can deduce one is the string scanf("%s", buffer); and the other is the buffer to read into. So, we are reading user input.
main	lea -0x20(%rbp), %rax	Same structure as the put

	mov %rax,%rsi mov $0x40072e,%edi mov $0x0,%eax call printf@plt	method in the above. It calls printf with three arguments, that would probably be printf("You entered: %s", buffer);

File Three: assignment3_3.o

Functions	Blocks of Assembly Code	Explanation of Functionality
AddNumbers	push %rbp mov %rsp, %rbp	Save callers frame pointer, create new stack frame.
AddNumbers	mov %edi, -0x4(%rbp) move %esi, -0x8(%rbp)	Creates two ints and sets them to the first and second arguments to the function by our special argument registers, %edi & %esi.
AddNumbers	mov -0x8(%rbp),%eax mov -0x4(%rbp),%edx	Loads the variables into registers %eax & %edx.
AddNumbers	add %edx,%eax	Adds the two registers and saves it to register %eax. This register will later hold on to the value as the function returns to the caller.
main	mov -0x8(%rbp),%edx mov -0xc(%rbp),%eax mov %edx,%esi mov %eax,%edi call AddNumbers	Previous assembly instruction sets up the frame and local variables as above assignment files. Here we use the source values from %rbp offsets for the two integers to call the function.
main	mov -0x8(%rbp),%edx mov -0xc(%rbp),%eax mov -0x4(%rbp),%ecx mov %eax,%esi mov $0x40074d,%edi mov $0x0,%eax call printf@plt	Sets up the variables and calls the printf function that alerts the user to enter two numbers.

File Four: assignment3_4.o

Southern New Hampshire University

Functions	Blocks of Assembly Code	Explanation of Functionality
PrintFact	mov %edi, -0x14(%rbp) movl $0x1, -0x4(%rbp) mov -0x14(%rbp), %eax mov %eax, -0x8(%rbp)	Stores argument 'n', and initializes a variable to a constant 1. Then we set the loop variable to our function parameter value found at register eax.
PrintFact	cmpl $0x0, -0x8(%rbp) jg 400647 <PrintFact+0x1a>	This compares the value at negative 8 offset of the base pointer which means while our value is not equal to zero.
PrintFact	imul -0x8(%rbp), %eax	Multiplies our factorial starting from the i and saving it to our initialized variable
PrintFact	subl $0x1, -0x8(%rbp)	Decrements our loop variable.
PrintFact	mov -0x4(%rbp), %eax	returns the value in eax.
PrintSum	add %eax, -0x4(%rbp)	This function is nearly identical except for this line that adds instead of multiplies.
DisplayMenu	mov $0x400851, %edi call 4004e0 <puts@plt>	This function has multiple put calls which prints the menu. ********** 1. Factorial 2. Summation 3. Quit **********
main	movl $0x0, -0x8(%rbp)	Initializes the choice variable
main	cmp $0x3, %eax jne 400728	Compares eax to the constant three and jumps if not equal.
main	cmp $0x1, %eax call PrintFact	Compares the value to a constant('1') and calls PrintFact function if true
main	cmp $0x2, %eax call PrintSum	Calls print sum if register(choice) eax is a value of two.